



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS
PROFESIONALES
ZONA MEDIA

INGENIERÍA MECATRÓNICA



PRÁCTICA 13

Control de Velocidad y Giro de Motor Usando Interfaz Gráfica

MATERIA: MICROCONTROLADORES

DOCENTE: Ing. JESÚS PADRÓN

ALUMNO: CARLOS ANGEL GARCÍA SIFUENTES

CLAVE: A383917

CARRERA: INGENIERÍA MECATRÓNICA

SEMESTRE: 4° SEMESTRE

FECHA: 24 DE MAYO DE 2026

PWM y Protocolo Serial desde Interfaz Gráfica en Python

CARLOS ÁNGEL GARCÍA SIFUENTES

a383917@alumnos.uaslp.mx

Fecha: 20 de mayo de 2026

Resumen—Se presenta el desarrollo de la Práctica 13, que integra control PWM de velocidad, lógica de puente H con cuatro transistores 2N2222A, comunicación serial USB y una interfaz gráfica en Python con `tkinter` para el control bidireccional de un motor de corriente directa. La Raspberry Pi Pico W recibe comandos de texto (`A<vel>`, `R<vel>`, `S`) desde la computadora a través del puerto USB-serial, los interpreta y genera las señales PWM y de habilitación necesarias para controlar la velocidad (0–100 %) y la dirección de giro (adelante/reversa) del motor. La interfaz gráfica proporciona un slider de velocidad, botones de dirección y un log serial en tiempo real.

I. INTRODUCCIÓN

I-A Objetivo e Importancia

Esta práctica representa la integración más compleja del curso: combina la generación de señales PWM, el control de potencia mediante transistores, la comunicación serial bidireccional y el desarrollo de una interfaz gráfica de usuario (GUI). El resultado es un sistema de control de motor DC completo, comparable a los drives industriales de baja potencia, donde la lógica de control reside en el software y la etapa de potencia está aislada mediante transistores.

La importancia del PWM en el control de motores radica en su eficiencia energética: a diferencia de un divisor resistivo, el transistor en modo conmutado prácticamente no disipa potencia en los estados de saturación y corte, entregando al motor exactamente la energía proporcional al ciclo de trabajo.

I-B Puente H y Control Bidireccional

Un **puente H** es un circuito de cuatro interruptores (en este caso transistores NPN 2N2222A) dispuestos en forma de H alrededor del motor. Al activar diagonalmente pares de transistores se invierte la polaridad aplicada al motor, cambiando su sentido de giro. La Tabla I muestra la lógica de activación.

Tabla I: Lógica de activación del puente H

Modo	Q1	Q2	Q3	Q4	Resultado
Adelante	PWM	OFF	OFF	ON	Motor gira adelante
Reversa	OFF	PWM	ON	OFF	Motor gira en reversa
Paro	OFF	OFF	OFF	OFF	Motor se detiene

Q1 y Q2 reciben la señal PWM (pines GP14 y GP16 respectivamente), mientras que Q3 y Q4 son salidas digitales simples (GP15 y GP17). Solo un par diagonal se activa a la vez; activar ambos simultáneamente cortocircuitaría la fuente (*shoot-through*).

I-C Protocolo de Comunicación Serial

La interfaz gráfica y el microcontrolador se comunican mediante un protocolo de texto simple sobre el puerto USB-serial a 115 200:

Tabla II: Protocolo de comandos serial

Comando	Formato	Ejemplo
Adelante	<code>A<vel>\n</code>	<code>A075\n</code> = adelante al 75 %
Reversa	<code>R<vel>\n</code>	<code>R050\n</code> = reversa al 50 %
Paro	<code>S\n</code>	detener el motor

II. DESARROLLO

II-A Material y Recursos Utilizados

- **Hardware:** 1 Placa Raspberry Pi Pico W, 1 motor DC de 5V–12V, 4 transistores NPN 2N2222A, 4 resistencias de 1 k Ω (bases), 4 diodos de protección 1N4007 (*flyback*), fuente de alimentación externa para el motor, 1 protoboard y alambre de conexión.
- **Software:** SDK de Raspberry Pi Pico, Pico – Visual Studio Code, Python 3 con `tkinter` (incluido en la instalación estándar) y `pyserial` (`pip install pyserial`).

II-B Asignación de Pines

Tabla III: Asignación de pines GPIO – Práctica 13

GPIO	Función	Tipo	Transistor
GP14	PWM adelante	PWM salida	Q1 (brazo superior)
GP15	Habilitación Q3	Digital OUT	Q3 (brazo inferior)
GP16	PWM reversa	PWM salida	Q2 (brazo superior)
GP17	Habilitación Q4	Digital OUT	Q4 (brazo inferior)

II-C Estructura del Proyecto y CMake

ARCHIVO CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)
set(PICO_BOARD pico_w)
include(pico_sdk_import.cmake)

project(PRACTICA_13)

pico_sdk_init()

add_executable(PUENTE
    PUENTE.c
)

target_link_libraries(PUENTE
    pico_stdlib
    hardware_adc
    hardware_pwm
    hardware_gpio
    hardware_clocks
)

pico_enable_stdio_usb(PUENTE 1)
pico_enable_stdio_uart(PUENTE 0)

pico_add_extra_outputs(PUENTE)
```

II-D Implementación del Firmware en C – PUENTE.c

CÓDIGO FUENTE – PUENTE.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "pico/stdlib.h"
#include "hardware/pwm.h"
#include "hardware/gpio.h"
#include "hardware/clocks.h"

#define PIN_Q1 14 // PWM adelante
#define PIN_Q3 15 // Enable Q3
#define PIN_Q2 16 // PWM reversa
#define PIN_Q4 17 // Enable Q4

#define PWM_FREQ_HZ 1000
```

```
#define PWM_MAX_DUTY 1000
#define DEAD_TIME_MS 200
#define KICKSTART_MS 80
#define KICKSTART_UMBRA 85
#define BUF_SIZE 16

typedef enum { MOTOR_ADELANTE, MOTOR_ATRAS,
    MOTOR_PARO } motor_dir_t;
static motor_dir_t dir_actual = MOTOR_PARO;

static void pwm_init_pin(uint pin) {
    gpio_set_function(pin, GPIO_FUNC_PWM);
    uint slice = pwm_gpio_to_slice_num(pin);
    ;
    uint chan = pwm_gpio_to_channel(pin);
    float clk_div = (float)clock_get_hz(
        clk_sys)
        / (PWM_FREQ_HZ *
            PWM_MAX_DUTY);
    pwm_config cfg = pwm_get_default_config(
        );
    pwm_config_set_clkdiv(&cfg, clk_div);
    pwm_config_set_wrap(&cfg, PWM_MAX_DUTY
        - 1);
    pwm_init(slice, &cfg, true);
    pwm_set_chan_level(slice, chan, 0);
}

static void set_pwm_duty(uint pin, uint
    duty) {
    if (duty > PWM_MAX_DUTY) duty =
        PWM_MAX_DUTY;
    pwm_set_chan_level(
        pwm_gpio_to_slice_num(pin),
        pwm_gpio_to_channel(
            pin), duty);
}

// Remapeo 1-100% del GUI a 35-100% para
// garantizar arranque
static uint duty_from_percent(int pct) {
    if (pct <= 0) return 0;
    int real = 35 + (pct * 65 / 100);
    if (real > 100) real = 100;
    return (uint)(real * PWM_MAX_DUTY /
        100);
}

static void motor_set(motor_dir_t dir, uint
    duty) {
    if (dir != dir_actual) { // Dead
        -time al cambiar direccin
        set_pwm_duty(PIN_Q1, 0);
        set_pwm_duty(PIN_Q2, 0);
        gpio_put(PIN_Q3, 0); gpio_put(
            PIN_Q4, 0);
        sleep_ms(DEAD_TIME_MS);
        dir_actual = dir;
    }
    if (dir == MOTOR_PARO) {
        gpio_put(PIN_Q3, 0); gpio_put(
```

```

        PIN_Q4, 0);
    set_pwm_duty(PIN_Q1, 0);
    set_pwm_duty(PIN_Q2, 0);
    return;
}

uint pin_pwm = (dir == MOTOR_ADELANTE)
    ? PIN_Q1 : PIN_Q2;
uint pin_on  = (dir == MOTOR_ADELANTE)
    ? PIN_Q4 : PIN_Q3;
uint pin_off = (dir == MOTOR_ADELANTE)
    ? PIN_Q3 : PIN_Q4;
gpio_put(pin_off, 0);
gpio_put(pin_on, 1);
if (duty < (KICKSTART_UMBRAL *
    PWM_MAX_DUTY / 100)) {
    set_pwm_duty(pin_pwm, PWM_MAX_DUTY)
        ; // Kick-start
    sleep_ms(KICKSTART_MS);
}
set_pwm_duty(pin_pwm, duty);
}

static void procesar_comando(const char *
cmd) {
    if (cmd[0] == 'A' || cmd[0] == 'a') {
        int pct = atoi(&cmd[1]);
        motor_set(MOTOR_ADELANTE,
            duty_from_percent(pct));
        printf("OK ADELANTE %d%%\n", pct);
    } else if (cmd[0] == 'R' || cmd[0] == '
r') {
        int pct = atoi(&cmd[1]);
        motor_set(MOTOR_ATRAS,
            duty_from_percent(pct));
        printf("OK REVERSA %d%%\n", pct);
    } else if (cmd[0] == 'S' || cmd[0] == '
s') {
        motor_set(MOTOR_PARO, 0);
        printf("OK PARO\n");
    } else {
        printf("ERR comando desconocido: %s
\n", cmd);
    }
    fflush(stdout);
}

int main(void) {
    stdio_init_all();
    while (!stdio_usb_connected()) sleep_ms
        (100); // Espera USB

    pwm_init_pin(PIN_Q1);
    pwm_init_pin(PIN_Q2);
    gpio_init(PIN_Q3); gpio_set_dir(PIN_Q3,
        GPIO_OUT); gpio_put(PIN_Q3, 0);
    gpio_init(PIN_Q4); gpio_set_dir(PIN_Q4,
        GPIO_OUT); gpio_put(PIN_Q4, 0);

    printf("--- PUENTE H LISTO ---\n");
    printf("Protocolo: A<vel> | R<vel> | S\
n");

```

```

    fflush(stdout);

    char    buf[BUF_SIZE];
    uint8_t idx = 0;
    while (true) {
        int c = getchar_timeout_us(0);
        if (c == PICO_ERROR_TIMEOUT) {
            sleep_ms(5); continue; }
        if (c == '\n' || c == '\r') {
            if (idx > 0) { buf[idx] = '\0';
                procesar_comando(buf); idx
                    = 0; }
        } else {
            if (idx < BUF_SIZE - 1) buf[idx
                ++] = (char)c;
        }
    }
    return 0;
}

```

II-D1. Configuración del PWM a 1kHz con resolución de 1000 niveles

A diferencia de prácticas anteriores donde se usó `pwm_set_wrap(slice, 255)`, aquí se configura el PWM con `PWM_MAX_DUTY = 1000` niveles para mayor precisión. El divisor de reloj se calcula automáticamente a partir de la frecuencia del sistema:

$$\text{clk_div} = \frac{f_{\text{sys}}}{f_{\text{PWM}} \times \text{PWM_MAX_DUTY}} = \frac{125 \text{ MHz}}{1 \text{ kHz} \times 1000} = 125 \quad (1)$$

II-D2. Dead-time y protección contra shoot-through

Cuando el comando cambia de dirección, la función `motor_set()` apaga todos los transistores y espera 200ms antes de activar la nueva diagonal. Este *dead-time* es esencial para evitar el *shoot-through*: el instante en que Q1 y Q3 (o Q2 y Q4) están simultáneamente activos, cortocircuitando la fuente de alimentación y destruyendo los transistores.

II-D3. Remapeo y kick-start

El remapeo de velocidad (`duty_from_percent`) convierte el rango de la GUI (1–100 %) al rango efectivo (35–100 %), ya que la mayoría de motores DC tienen una tensión mínima de arranque. Para velocidades bajas (por debajo del 85 % del máximo) se aplica además un pulso de *kick-start* al 100 % durante 80 ms, suficiente para que el rotor supere la inercia estática y el motor arranque de forma confiable.

II-E Interfaz Gráfica en Python – MONITOR.py

La GUI se construyó con `tkinter`, la biblioteca gráfica estándar de Python, sin dependencias externas adicionales.

les más allá de pyserial.

FRAGMENTO CLAVE – MONITOR.py (comandos y conexión serial)

```
import tkinter as tk
from tkinter import ttk, messagebox
import serial, serial.tools.list_ports
import threading, time

def _cmd_adelante(self):
    v = self.speed_var.get()
    self._enviar(f"A{v:03d}")
    self._set_estado("ADELANTE", v)

def _cmd_reversa(self):
    v = self.speed_var.get()
    self._enviar(f"R{v:03d}")
    self._set_estado("REVERSA", v)

def _cmd_paro(self):
    self._enviar("S")
    self._set_estado("PARO", 0)
    self.speed_var.set(0)

def _enviar(self, cmd: str):
    if not self.serial_port or not self.serial_port.is_open:
        self._log(f"SIN CONEXION comando ignorado: {cmd}")
        return
    self.serial_port.write((cmd + "\n").encode())
    self._log(f"TX -> {cmd}")

def _leer_serial(self):
    # Hilo de fondo: lee respuestas del Pico
    while self.serial_port and self.serial_port.is_open:
        line = self.serial_port.readline() \
            .decode(errors="replace") \
            .strip()
        if line:
            self.after(0, self._log, f"RX <- {line}")
```

La interfaz cuenta con un selector de puerto COM con botón de actualización, selector de baud rate, slider de velocidad (0–100 %), tres botones (*Adelante*, *Paro*, *Reversa*), un display de estado en tiempo real y un log serial con marcas de tiempo. La lectura de respuestas del Pico se ejecuta en un hilo de fondo (`threading.Thread`), evitando que el bucle de eventos de `tkinter` se bloquee.

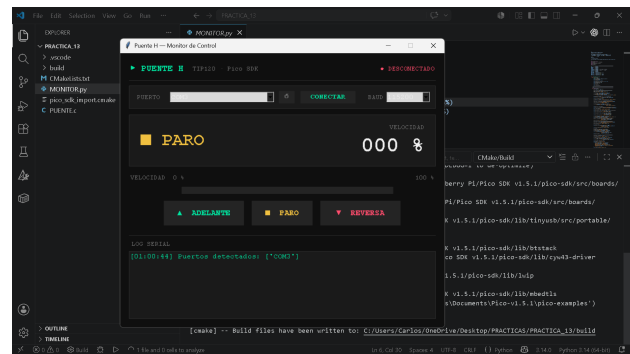


Figura 1: Interfaz gráfica `MONITOR.py`: panel de conexión serial, display de estado, slider de velocidad, botones de control y log serial en tiempo real.

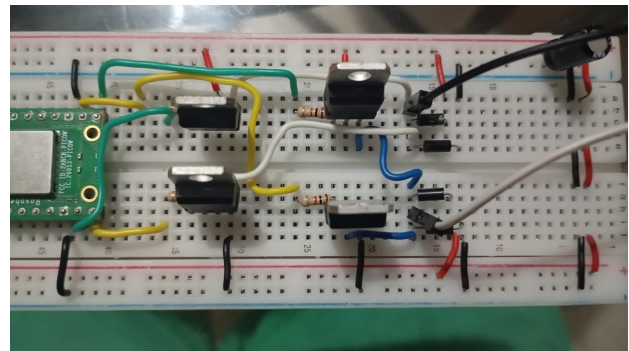


Figura 2: Esquemático del puente H con cuatro transistores 2N2222A, diodos de protección flyback 1N4007 y motor DC.

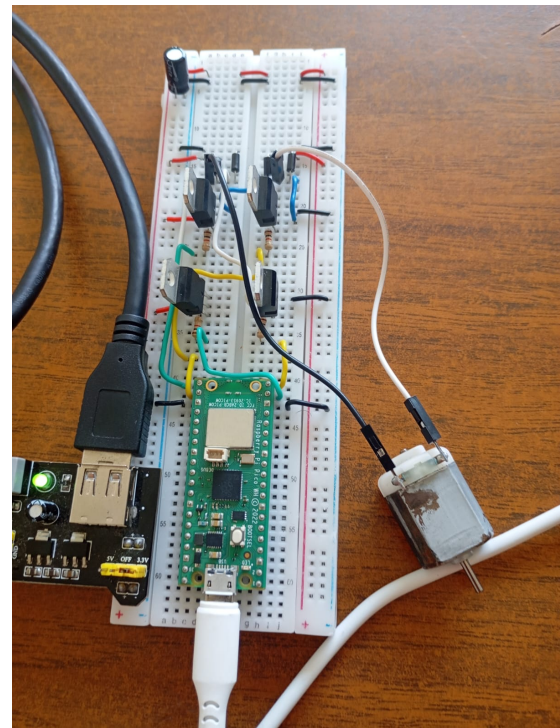


Figura 3: Circuito físico en funcionamiento con el puente H en protoboard y el motor DC girando.

III. PREGUNTAS DE REPASO

III-A ¿Qué es el ciclo de trabajo en un PWM?

El ciclo de trabajo (*duty cycle*) es la fracción del período de la señal PWM durante la cual la salida permanece en estado alto, expresada como porcentaje:

$$D = \frac{t_{ON}}{T} \times 100 \%$$

Representa directamente la potencia promedio entregada a la carga: un ciclo de trabajo del 0 % equivale a cero potencia (motor detenido), mientras que el 100 % entrega la potencia máxima (velocidad máxima del motor).

III-B ¿Qué significa que el ciclo de trabajo del PWM sea de 40 %?

Un ciclo de trabajo del 40 % significa que en cada período de la señal PWM, la salida permanece en estado alto durante el 40 % del tiempo y en bajo el 60 % restante. Con la configuración de esta práctica (1 kHz, $T = 1 \text{ ms}$), la señal estaría en alto 0.4 ms y en bajo 0.6 ms por ciclo. El motor recibiría en promedio el 40 % de la tensión de alimentación, girando aproximadamente al 40 % de su velocidad máxima (en la práctica, al 40 % del rango remapeado de 35–100 %).

III-C ¿Cómo se genera una señal PWM en un microcontrolador?

El RP2040 de la Raspberry Pi Pico W genera el PWM mediante un contador de hardware que se incrementa con cada ciclo del reloj del sistema. El contador compara su valor con dos registros: *wrap* (valor máximo, determina la frecuencia) y *level* (valor de comparación, determina el duty cycle). Cuando el contador es menor que *level*, la salida está en alto; cuando lo supera, baja. Al alcanzar *wrap*, el contador reinicia. En el código esto se configura con `pwm_config_set_wrap()`, `pwm_config_set_clkdiv()` y `pwm_set_chan_level()`, sin ninguna carga adicional para el procesador una vez iniciado.

III-D ¿Por qué no puede conectarse directamente el PWM del microcontrolador al motor?

Porque los pines GPIO del RP2040 tienen una capacidad de corriente máxima de 12 mA (4 mA recomendados). Un motor DC, incluso pequeño, puede requerir cientos de miliamperes o varios amperes durante el arranque. Conectar el motor directamente al GPIO

dañaría permanentemente el microcontrolador, ya que la corriente demandada superaría ampliamente el límite seguro del silicio. Por ello es necesario interponer una etapa de potencia (en este caso el puente H con transistores) que aísla eléctricamente el microcontrolador y amplifica la corriente de control a los niveles requeridos por el motor.

III-E ¿Cuánta corriente puede entregar el pin PWM del microcontrolador?

El RP2040 especifica una corriente máxima absoluta de 12 mA por pin GPIO individualmente, con un límite global de 50 mA para todos los pines del banco de E/S. La corriente recomendada para operación segura y estable es de 4 mA por pin. Esto es suficiente para polarizar la base de un transistor NPN (que requiere solo 1 mA–2 mA) pero completamente insuficiente para alimentar directamente una carga inductiva como un motor DC.

III-F ¿Qué función tiene el puente H en el proyecto?

El puente H cumple dos funciones fundamentales simultáneamente. La primera es la **amplificación de corriente**: convierte la señal de control del microcontrolador (máx. 12 mA) en una señal capaz de alimentar el motor (cientos de mA), usando los transistores 2N2222A como interruptores de potencia. La segunda es el **control bidireccional**: al activar diagonalmente pares alternos de transistores (Q1+Q4 para adelante, Q2+Q3 para reversa), invierte la polaridad aplicada a los terminales del motor, cambiando su sentido de giro. Sin el puente H solo sería posible encender y apagar el motor, pero no invertir su dirección. Los diodos de protección *flyback* integrados absorben los picos de tensión generados por la inductancia del bobinado del motor al conmutar, protegiendo los transistores.

IV. CONCLUSIONES

- La arquitectura cliente-servidor del sistema (GUI en Python ↔ Pico W por serial) demostró ser una solución elegante para separar responsabilidades: la interfaz gráfica maneja la interacción con el usuario, mientras el firmware maneja el hardware en tiempo real. Cualquier mejora en la GUI no requiere recompilar el firmware y viceversa.
- El *dead-time* de 200 ms al cambiar dirección es una protección crítica contra el *shoot-through* en el puente H. Sin esta pausa, al cambiar de adelante a

reversa podrían activarse simultáneamente transistores de la misma rama, cortocircuitando la fuente y destruyendo los 2N2222A.

- El *kick-start* de 80 ms al 100 % al arrancar a baja velocidad resolvió el problema de inercia estática del motor, que en pruebas sin este pulso no arrancaba por debajo del 40 % de duty cycle efectivo. Esta técnica es común en drivers comerciales de motores DC.
- La función `duty_from_percent()` que remapea el rango de la GUI (1–100 %) al rango efectivo del motor (35–100 %) es un ejemplo de calibración de sistema: los valores nominales del control no siempre corresponden linealmente al comportamiento físico del actuador.
- Como mejora futura se podría añadir un encoder de cuadratura al motor para implementar control en lazo cerrado (PID), haciendo que la velocidad real del eje coincida con la velocidad comandada independientemente de la carga mecánica, elevando el sistema al nivel de un servo-drive real.

ANEXOS

A. Firmware – PUENTE.c (completo)

ANEXO A – PUENTE.c

```
// Ver sección de Desarrollo para el
// código completo con comentarios.
// Archivo disponible en el repositorio
// GitHub:
// github.com/CAGS/
// Practical3_Microcontroladores
```

B. Interfaz Gráfica – MONITOR.py (completo)

ANEXO B – MONITOR.py

```
# Ver sección de Desarrollo para los
# fragmentos clave comentados.
# Archivo completo disponible en el
# repositorio GitHub.
# Dependencias: pip install pyserial
# Ejecución: python MONITOR.py
```

C. Archivo CMakeLists.txt

ANEXO C – CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)
set(PICO_BOARD pico_w)
include(pico_sdk_import.cmake)
```

```
project(PRACTICA_13)
pico_sdk_init()
add_executable(PUENTE PUENTE.c)
target_link_libraries(PUENTE
    pico_stdlib hardware_adc hardware_pwm
    hardware_gpio hardware_clocks)
pico_enable_stdio_usb(PUENTE 1)
pico_enable_stdio_uart(PUENTE 0)
pico_add_extra_outputs(PUENTE)
```

D. Diagrama en Protoboard

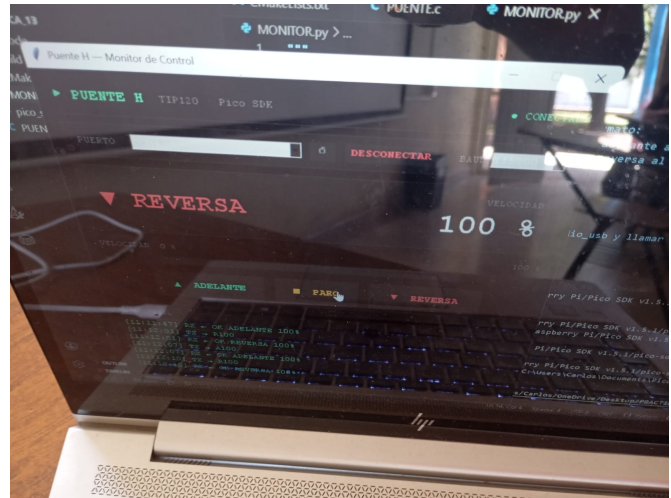


Figura 4: Pantalla de la computadora, donde se muestra el monitor trabajando